

# Arithmetic Operator

```
In [1]: x1, y1 = 10,5
```

```
In [2]: x1 ^ y1
```

```
Out[2]: 15
```

```
In [3]: x1 + y1
```

```
Out[3]: 15
```

```
In [4]: x1 - y1
```

```
Out[4]: 5
```

```
In [5]: x1 * y1
```

```
Out[5]: 50
```

```
In [6]: x1 / y1
```

```
Out[6]: 2.0
```

```
In [7]: x1 // y1
```

```
Out[7]: 2
```

```
In [8]: x1 % y1
```

```
Out[8]: 0
```

```
In [9]: x1 ** y1
```

```
Out[9]: 100000
```

```
In [10]: x2 = 3  
y2 = 3  
x2 ** y2
```

```
Out[10]: 27
```

# Assignment Operator

```
In [11]: x = 2
```

```
In [12]: x = x + 2 # if you want to increment by 2
```

```
In [13]: x
```

```
Out[13]: 4
```

```
In [14]: x += 2  
x
```

```
Out[14]: 6
```

```
In [15]: x += 2  
x
```

```
Out[15]: 8
```

```
In [16]: x *= 2
```

```
In [17]: x
```

```
Out[17]: 16
```

```
In [18]: x -= 2
```

```
In [19]: x
```

```
Out[19]: 14
```

```
In [20]: x /= 2  
x
```

```
Out[20]: 7.0
```

```
In [21]: x //= 2  
x
```

```
Out[21]: 3.0
```

```
In [22]: a, b = 5,6 # you can assigned variable in one line as well  
print(a)  
print(b)
```

```
5
```

```
6
```

```
In [23]: a = 5  
b = 6  
print(a)  
print(b)
```

```
5
```

```
6
```

```
In [24]: a
```

```
Out[24]: 5
```

```
In [25]: b
```

```
Out[25]: 6
```

## Unary Operator

- unary means 1 || binary means 2 - Here we are applying unary minus operator(-) on the operand n; the value of m becomes -7, which indicates it as a negative value.

```
In [26]: n = 7 #negattion
n
```

```
Out[26]: 7
```

```
In [27]: m = -(n)
m
```

```
Out[27]: -7
```

```
In [28]: n
```

```
Out[28]: 7
```

```
In [29]: -n
```

```
Out[29]: -7
```

## Relational operator

```
In [30]: # we are using this operator for comparing
```

```
In [31]: a = 5
b = 6
```

```
In [32]: a<b
```

```
Out[32]: True
```

```
In [33]: a>b
```

```
Out[33]: False
```

```
In [34]: # a = b # we cannot use = operator that means it is assigning
```

```
In [35]: a == b
```

Out[35]: False

In [36]: `a != b`

Out[36]: True

In [37]: `# hear if i change b = 6`  
`b = 5`

In [38]: `a == b`

Out[38]: True

In [39]: `a`

Out[39]: 5

In [40]: `b`

Out[40]: 5

In [41]: `a > b`

Out[41]: False

In [42]: `a >= b`

Out[42]: True

In [43]: `a <= b`

Out[43]: True

In [44]: `a < b`

Out[44]: False

In [45]: `a>b`

Out[45]: False

In [46]: `b = 7`

In [47]: `a != b`

Out[47]: True

## Logical Operator

```
In [48]: a = 5  
b = 4
```

```
In [49]: a < 8 and b < 5 #refer to the truth table
```

```
Out[49]: True
```

```
In [50]: a < 8 and b < 2
```

```
Out[50]: False
```

```
In [51]: a < 8 or b < 2
```

```
Out[51]: True
```

```
In [52]: a>8 or b<2
```

```
Out[52]: False
```

```
In [53]: x = False  
x
```

```
Out[53]: False
```

```
In [54]: not x # you can reverse the operation
```

```
Out[54]: True
```

```
In [55]: x = not x  
x
```

```
Out[55]: True
```

```
In [56]: x
```

```
Out[56]: True
```

```
In [57]: not x
```

```
Out[57]: False
```

## Number System Coverstion (bit-binary digit)

```
In [58]: 25
```

```
Out[58]: 25
```

```
In [59]: bin(25)
```

Out[59]: '0b11001'

In [60]: `int(0b11001)`

Out[60]: 25

In [61]: `bin(30)`

Out[61]: '0b11110'

In [62]: `int(0b11110)`

Out[62]: 30

In [63]: `int(0b11001)`

Out[63]: 25

In [64]: `oct(25)`

Out[64]: '0o31'

In [65]: `int(0o31)`

Out[65]: 25

In [66]: `int(0b11110)`

Out[66]: 30

In [67]: `0o31`

Out[67]: 25

In [68]: `0b11001`

Out[68]: 25

In [69]: `int(0b11001)`

Out[69]: 25

In [70]: `bin(7)`

Out[70]: '0b111'

In [71]: `hex(25)`

Out[71]: '0x19'

In [72]: `0x19`

Out[72]: 25

In [73]: `hex(16)`

Out[73]: '0x10'

In [74]: `0xa`

Out[74]: 10

In [75]: `0xb`

Out[75]: 11

In [76]: `hex(1)`

Out[76]: '0x1'

In [77]: `hex(25)`

Out[77]: '0x19'

In [78]: `0x19`

Out[78]: 25

In [79]: `0x15`

Out[79]: 21

## Swap 2-Variable in Python

In [80]: *#(a,b = 5,6) After swap we should get ==> (a, b = 6,5 )*

In [81]: `a = 5`  
`b = 6`

In [82]: `a = b`  
`b = a`

In [83]: `print(a)`  
`print(b)`

6  
6

In [84]: *# in above scenario we lost the value 5*  
`a1 = 7`  
`b1 = 8`

```
In [85]: temp = a1
        a1 = b1
        b1 = temp
```

```
In [86]: print(a1)
        print(b1)
```

8

7

- in the above code we are using third variable

```
In [87]: a2 = 5
        b2 = 6
```

```
In [88]: #swap variable formulas without using 3rd formul
        a2 = a2 + b2 # 5+6 = 11
        b2 = a2 - b2 # 11-6 = 5
        a2 = a2 - b2 # 11-5 = 6
```

```
In [89]: print(a2)
        print(b2)
```

6

5

```
In [90]: 0b110
```

```
Out[90]: 6
```

```
In [91]: 0b101
```

```
Out[91]: 5
```

```
In [92]: print(0b110)
        print(0b101)
```

6

5

```
In [93]: # but when we use a2 + b2 then we get 11 that means we will get 4 bit which is 1 bi
        print(bin(11))
        print(0b1011)
```

0b1011

11

- XOR

```
In [94]: print(a2)
        print(b2)
```

6

5



```
In [95]: # there is other way to work using swap variable also which is XOR because it will
a2 = a2 ^ b2
b2 = a2 ^ b2
a2 = a2 ^ b2
```

```
In [96]: print(a2)
print(b2)
```

5  
6

```
In [97]: a2, b2
```

```
Out[97]: (5, 6)
```

```
In [98]: a2 , b2 = b2, a2    # how it work is b2 6 a2 is 5 first it goes into stack & then i
```

```
In [99]: print(a2)
print(b2)
```

6  
5

## Bitwise Operator

- WE HAVE 6 OPERATORS COMPLEMENT (~) || AND (&) || OR (|) || XOR (^) || LEFT SHIFT (<<) || RIGHT SHIFT (>>)

```
In [100... print(bin(12))
print(bin(13))
```

0b1100  
0b1101

```
In [101... 0b1101
```

```
Out[101... 13
```

```
In [102... 0b1100
```

```
Out[102... 12
```

## Complement (~) (Tilde or Tild)

~12 # why we get -13 . first we understand what is complment means (reversr of binary format)

```
In [103... ~12
```

```
Out[103... -13
```

```
In [104... ~46
```

Out[104...] -47

In [105...] ~54

Out[105...] -55

In [106...] ~10

Out[106...] -11

## Bit Wise and Operator

AND - LOGICAL OPERATOR `|||` & - BITWISE AND OPERATOR (we know that 1 & 1 is 1) 12 - 00001100 13 - 00001101 when we are add both then then outut we will get as 12

In [107...] 12 & 13

Out[107...] 12

In [108...] 12 | 13

Out[108...] 13

In [109...] 1 & 0

Out[109...] 0

In [110...] 1 | 0

Out[110...] 1

In [111...] bin(13)

Out[111...] '0b1101'

In [112...] print(bin(35))  
print(bin(40))

0b100011

0b101000

In [113...] 35 & 40

Out[113...] 32

In [114...] 35 | 40

Out[114...] 43

In [115...] *# in XOR if the both number are different then we will get 1 or else we will get 0*  
12 ^ 13

Out[115... 1

```
In [116... print(bin(25))  
print(bin(30))
```

```
0b11001  
0b11110
```

```
In [117... 25^30
```

Out[117... 7

```
In [118... bin(7)
```

Out[118... '0b111'

```
In [119... bin(25)
```

Out[119... '0b11001'

```
In [120... bin(30)
```

Out[120... '0b11110'

```
In [121... 0b00111
```

Out[121... 7

```
In [122... bin(10)
```

Out[122... '0b1010'

```
In [123... 10<<1
```

Out[123... 20

```
In [124... 10<<2
```

Out[124... 40

```
In [125... bin(10)
```

Out[125... '0b1010'

```
In [126... 10<<1
```

Out[126... 20

```
In [127... 10<<2
```

Out[127... 40

```
In [128... # BIT WISE LEFT SHIFT OPERATOR
# in left shift what we need to to we need shift in left hand side & need to shift
#bit wise left operator bydefault you will take 2 zeros ( )
#10 binary operator is 1010 | also i can say 1010
```

```
In [129... 10<<2
```

```
Out[129... 40
```

```
In [130... bin(20)
```

```
Out[130... '0b10100'
```

## BITWISE RIGHTSHIFT OPERATOR

- left side we are gaining the bits
- right side we are lossing bits

```
In [131... bin(10)
```

```
Out[131... '0b1010'
```

```
In [132... 10 >> 1
```

```
Out[132... 5
```

```
In [133... 10>>2
```

```
Out[133... 2
```

```
In [134... bin(20)
```

```
Out[134... '0b10100'
```